SENTINEL AI – ENTERPRISE AI GATEWAY

A PROJECT REPORT

Submitted by

JEFFREY ANTONY J (2116241901041)
JOE BENEDICT A (2116241901042)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "SENTINEL AI – ENTERPRISE AI GATEWAY PROJECT" is the bonafide work of "JEFFREY ANTONY J (2116241901041), JOE BENEDICT A (2116241901042) " who carried out the project work under my supervision.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

SentinelAI is proposed as a centralized Enterprise AI Gateway for applications that use Large Language Models (LLMs), including services from OpenAI, Google Gemini and Anthropic Claude. In a direct-integration model, every application can send each prompt to a provider independently. Repeated or similar prompts may therefore create redundant API requests, increase token consumption and operational cost, and make usage difficult to govern across an organization.

The proposed platform places a managed gateway between enterprise applications and model providers. It brings together authentication, organization-based management, policy enforcement, provider routing, audit logging, usage analytics and cost monitoring. Its central design element is Enterprise Memory Intelligence: before a new provider request is made, the system checks whether a similar verified response can be reused. When a verified match exists, the response can be returned without a fresh model call. When no match exists, the request is routed to the selected provider; the generated response is to be validated and stored for potential reuse.

The review material identifies Python 3.13+, Django, Django REST Framework and React.js/Bun as the application stack, with PostgreSQL and SQLite listed as database options. The scope includes organization-aware access, memory-assisted response handling, multi-provider integration, monitoring and a dashboard. At Review 1, literature survey, problem identification, requirement analysis and methodology, and the proposed architecture and core modules were marked complete. Implementation, testing and final evaluation were still pending. Accordingly, reduced API usage, lower costs, faster repeated responses and improved consistency are intended outcomes, not yet measured results. The report documents the problem, related work, proposed design, security considerations, methodology, current status and evaluation plan.

Keywords: Enterprise AI Gateway, Large Language Models, Enterprise Memory Intelligence, response reuse, AI governance, audit logging, usage analytics.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

JEFFREY ANTONY J (2116241901041)
JOE BENEDICT A (2116241901042)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	2
	1.4 SCOPE	2
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	2
	1.6 SDG ALIGNMENT	3
2	**LITERATURE REVIEW**	**4**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	4
	2.1.1 SUMMARY OF LITERATURE	4
	2.1.2 COMPARISON OF EXISTING AND PROPOSED APPROACH	5
	2.2 RESEARCH GAPS	5
	2.3 PROPOSED SYSTEM	5
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	5
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	6
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	6
3	**SYSTEM DESIGN**	**7**
	3.1 SYSTEM ARCHITECTURE	7
	3.2 MODULE DESIGN	7
	3.3 DATA FLOW / UML DIAGRAMS	8
	3.3.1 CONTEXT-LEVEL DATA FLOW	8
	3.3.2 REQUEST PROCESSING ACTIVITY FLOW	9
	3.3.3 SEQUENCE OF GATEWAY INTERACTIONS	9

	3.4	THREAT MODEL AND ATTACK SURFACE	10
	3.5	SECURITY CONTROLS	11
4	**SYSTEM DESCRIPTION**		**12**
	4.1	DATA COLLECTION	12
	4.2	DATA PREPROCESSING	12
	4.2.1	ALGORITHMS / METHODOLOGY	13
	4.2.2	RESULTS AND DISCUSSION	14
5	**CONCLUSION**		**16**
	5.1	CONCLUSION	16
	5.2	FUTURE WORK	16
	REFERENCES		**17**
	APPENDIX A - SYSTEM REQUIREMENTS AND EVALUATION PLAN		**18**
	APPENDIX B - PROJECT SCREENSHOTS		**20**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SECURITY REQUIREMENTS AND CONSIDERATIONS	3
1.2	SDG ALIGNMENT	3
2.2	COMPARISON OF DIRECT LLM INTEGRATION AND SENTINELAI	5
3.1	SENTINELAI MODULE DESIGN	8
3.2	THREAT MODEL AND ATTACK SURFACE	10
3.3	PROPOSED SECURITY CONTROLS	11
4.1	OPERATIONAL DATA ELEMENTS	12
4.2	PRELIMINARY EVALUATION CRITERIA	15
A.1	HARDWARE REQUIREMENTS	18
A.2	SOFTWARE REQUIREMENTS	18

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	SENTINELAI SYSTEM ARCHITECTURE	7
3.2	CONTEXT-LEVEL DATA FLOW DIAGRAM	8
3.3	REQUEST PROCESSING ACTIVITY FLOW	9
3.4	SEQUENCE DIAGRAM FOR REQUEST, MEMORY AND PROVIDER HANDLING	10
4.1	ENTERPRISE MEMORY INTELLIGENCE DECISION FLOW (CONCEPTUAL)	14
B.1	DASHBOARD ANALYTICS OVERVIEW	20
B.2	ENTERPRISE MEMORY INTELLIGENCE CACHE RECORDS	20
B.3	AUDIT LOG HISTORY	21
B.4	AUDIT TRANSACTION DETAILS	21
B.5	FINANCE ASSISTANT WORKSPACE	22
B.6	TECHNICAL ASSISTANT WORKSPACE	22

CHAPTER 1

INTRODUCTION

1.1 Background

Large Language Models (LLMs) are used in enterprise applications to support tasks such as question answering, text generation, summarisation and software-development assistance. Applications commonly connect to model-provider APIs to submit a prompt and receive a generated response. The review material identifies OpenAI, Google Gemini and Anthropic Claude as examples of providers that SentinelAI is intended to support.

When every application handles provider calls independently, repeated or similar requests can be sent multiple times. This increases API usage and token consumption, and it can lead to repeated operating costs. It also makes organisation-wide control difficult because authentication, policy decisions, monitoring and audit records may be scattered across separate applications.

SentinelAI is proposed as an Enterprise AI Gateway between enterprise applications and LLM providers. It centralises request handling and introduces Enterprise Memory Intelligence (EMI), which checks for a similar verified response before routing a request to a provider. If a suitable verified response exists, it can be reused; otherwise, the request is routed to a selected provider and the generated output is intended to be validated and stored. The design also includes organisation management, audit logging, usage analytics and cost monitoring.

1.2 Problem Statement

Direct LLM integration creates a fragmented operating model for organisations that use AI across multiple applications. Similar prompts may trigger redundant provider calls, consuming tokens and increasing cost. Without a shared management layer, organisations may also lack consistent authentication, policy enforcement, usage visibility, audit trails and the ability to reuse trusted responses across applications.

The problem addressed by SentinelAI is the absence of a common gateway that can mediate AI requests, apply organisation-aware controls, look for reusable verified responses and record how the service is used. The project seeks to design such a layer while supporting integration with multiple LLM providers.

1.3 Objectives

- Develop a centralized AI Gateway for enterprise applications that use LLM services.
- Reduce unnecessary LLM API calls through Enterprise Memory Intelligence and verified response reuse.
- Improve response consistency by reusing trusted, verified outputs when an appropriate match exists.
- Reduce API cost and token consumption for repeated or similar requests.
- Provide organization-based AI management and policy enforcement.
- Monitor AI usage through centralized analytics and cost tracking.
- Maintain audit logs for AI interactions and gateway activity.
- Support secure and scalable integration with multiple LLM providers.

1.4 Scope

The proposed scope covers request intake, authentication, organisation context, policy checks, Enterprise Memory Intelligence, provider selection and routing, response validation, storage of verified responses, audit logging, usage analytics and cost monitoring. An interactive dashboard is intended to present organisation-level activity and system usage.

The system is designed to sit between enterprise applications and external LLM APIs rather than to train a new foundation model. The review references do not specify a separate model-training dataset or a final similarity-matching algorithm. The quality of memory matching, verification policy, cache freshness and measured savings therefore remain matters for implementation and evaluation.

The scope excludes any claim of measured performance at this stage. Review 1 records architecture and planning activities as complete while implementation, testing and final evaluation remain pending. This report treats the gateway as a proposed system design and identifies evaluation activities needed to verify the intended outcomes.

1.5 Security Requirements / Security Considerations

Because SentinelAI mediates prompts, generated responses and provider credentials, security controls are part of the gateway design. The following items are requirements or design considerations; the review material does not establish that each control has already been implemented.

Table 1.1 Security requirements and considerations

Requirement	Design consideration
Authentication	Verify the identity of the calling user or application before processing an AI request.
Organisation isolation	Scope access, policies, memory lookups and analytics to the appropriate organisation.
Authorisation and policy	Check whether the user or application is allowed to use the selected provider or operation.
Secret protection	Keep provider API credentials out of client-side code and avoid recording secrets in audit logs.
Input and output handling	Validate request structure and define checks for provider responses before storing them as reusable verified outputs.
Memory safety	Avoid cross-organisation response leakage; define verification, freshness and invalidation rules for cached content.
Auditability	Record meaningful request outcomes and usage metadata while minimising sensitive prompt or response content in logs.
Abuse and availability	Plan rate limits, failure handling and provider-error logging to reduce misuse and improve operational visibility.

1.6 SDG Alignment

The project can be related to sustainable development through its emphasis on technology infrastructure and accountable use of AI services. This is a design-level alignment, not a claim that SentinelAI has already demonstrated an SDG impact.

Table 1.2 SDG alignment

SDG / goal	Relevant target	Project contribution / alignment
SDG 9: Industry, Innovation and Infrastructure	Target 9.5: enhance scientific research and technological capabilities.	SentinelAI proposes a shared gateway for integrating and managing LLM services across applications.
SDG 16: Peace, Justice and Strong Institutions	Target 16.6: develop effective, accountable and transparent institutions.	Organisation-aware controls, usage analytics and audit logs are intended to improve transparency and accountability in enterprise AI use.

CHAPTER 2

LITERATURE REVIEW

This chapter summarises the works cited in the project review material and relates them to the problem addressed by SentinelAI. The cited sources inform the context of LLM applications, retrieval, prompt-injection risk and orchestration. They do not, individually, specify the complete enterprise gateway proposed here.

2.1 Existing System with Security Approaches

In a direct-integration system, an enterprise application sends a prompt to an LLM provider API and returns the generated response to the user. The approach is straightforward and gives an application direct access to model capabilities. However, repeated prompts can trigger repeated API calls, and separate applications may implement authentication, policies, logging and usage reports differently.

Security measures in individual applications can provide local protection, but organisations may not have one point for enforcing policy, analysing usage and maintaining a shared audit trail across applications. Direct calls also limit the ability to reuse a verified response across different applications. SentinelAI proposes a common control layer rather than replacing the underlying LLM providers.

2.1.1 Summary of Literature

The literature reviewed provides context for the design of SentinelAI. Lewis et al. (2020) [1] describe Retrieval-Augmented Generation as a way to retrieve relevant external knowledge to support language-model responses. Willison (2023) [2] discusses prompt-injection risks, where untrusted instructions may influence LLM-based systems. Chase (2023) [3] documents LLM orchestration with LangChain, supporting structured application development. OpenAI’s GPT-4 technical report [4] and Anthropic’s Constitutional AI work [5] provide additional context on language-model capabilities and alignment. These works address individual methods and risks; they do not themselves provide the shared enterprise gateway, tenant scoping, verified-response reuse, audit and cost monitoring proposed in SentinelAI.

The review references also list OpenAI's GPT-4 Technical Report (2023) and Anthropic's Constitutional AI paper (2022). These sources are retained in the reference list as contextual materials identified in the zeroth-review document.

Table 2.2 Comparison of direct LLM integration and SentinelAI

Capability	Direct application-to-provider flow	SentinelAI proposed approach
Request path	Each application invokes a provider API directly.	Applications send requests through a central gateway.
Authentication and organisation control	Implemented separately by each application, where present.	Centralised authentication and organisation-based management are planned.
Repeated or similar requests	Can cause another provider API call each time.	Enterprise Memory Intelligence checks for a similar verified response first.
Provider choice	Typically configured in each application.	A provider router is designed to connect to OpenAI, Gemini and Claude.
Usage visibility	May be distributed across applications and provider accounts.	Central usage analytics, cost monitoring and audit logging are planned.
Evidence status	Describes the current direct-integration pattern.	Proposed; implementation and evaluation were pending at Review 1.

2.2 Research Gaps

- No shared request-management layer is described for applying common controls across multiple enterprise applications.
- Repeated or semantically similar requests may generate redundant API calls when responses are not reused across applications.
- Authentication, organisation policies and auditing can be fragmented when implemented independently by each application.
- Organisations need a consolidated view of AI usage, token consumption and operational cost.
- Reusing responses requires appropriate matching, verification and strict organisation boundaries.

2.3 Proposed System

2.3.1 Overview of the Proposed System

SentinelAI is designed as a centralized Enterprise AI Gateway between enterprise applications and model providers. A request enters the gateway, is associated with an authenticated user or application and organisation, and is checked against applicable policies. Enterprise Memory Intelligence then searches for a similar verified response. A match can be reused; if no match is found, the request is sent through the AI gateway and provider router to the selected provider. The returned output is intended to pass validation before it is stored for future reuse and returned to the caller.

Audit logging, usage analytics, cost monitoring and organisation governance are cross-cutting functions in the design. The interface is intended to present key usage information through a dashboard. The review material identifies these components as planned modules; it does not report final deployment measurements.

2.3.2 Objectives of the Proposed System

- Centralise AI access behind a common gateway.
- Check for verified response reuse before making a provider call.
- Integrate multiple LLM providers through a managed routing layer.
- Apply organisation-based authentication and policy controls.
- Validate and store responses that are suitable for reuse.
- Provide audit logs, usage analytics and cost monitoring.

2.3.3 Advantages of the Proposed System

- Potential reduction in redundant provider calls and token consumption for repeated or similar requests.
- Potentially faster delivery when a verified memory match can be reused.
- More consistent responses when the same verified output is suitable for the same request context.
- Centralised organisation management and a common point for policy enforcement.
- Consolidated auditing and usage visibility across participating applications.
- A provider-routing design that can accommodate multiple LLM services.

These advantages are design objectives. Their magnitude and practical reliability must be established through controlled tests after implementation.

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

The architecture separates request intake, access control, memory lookup, provider routing, response validation and monitoring. The user or enterprise application begins the interaction; authentication and organisation context are resolved before policy checks. The memory component decides whether a verified response can be reused. A miss proceeds to a provider router, while the resulting response is intended to be validated and recorded before delivery.

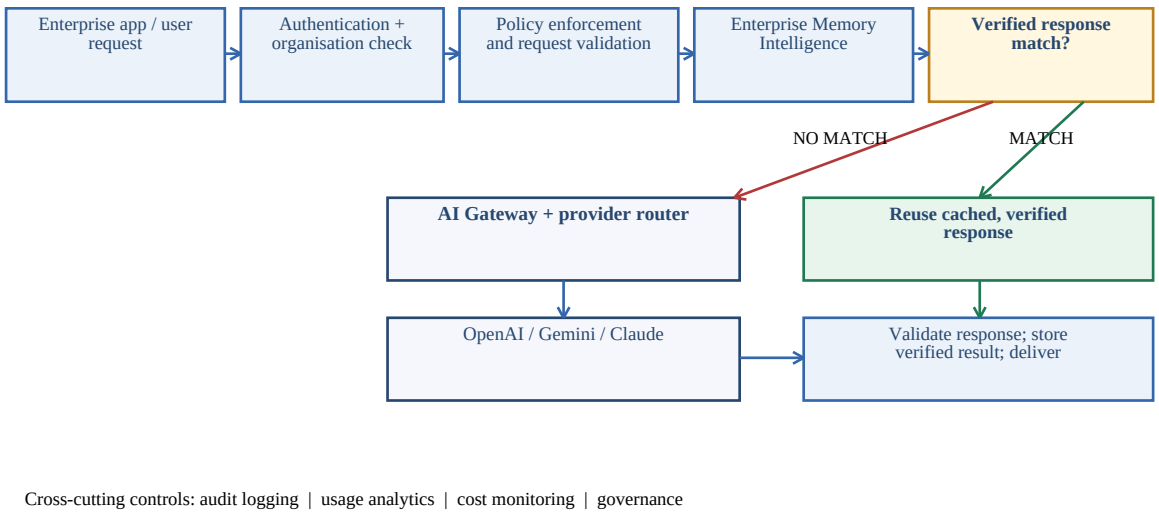


Figure 3.1 SentinelAI system architecture

The diagram presents the proposed end-to-end flow rather than a claim of completed implementation. The exact response-matching method, verification policy and rules for cache expiry need to be specified and tested during implementation.

3.2 Module Design

The review material divides SentinelAI into five core modules. The responsibilities below describe the intended design of each module.

Table 3.1 SentinelAI module design

Module	Main responsibilities	Expected inputs / outputs
1. Authentication and Organisation Management	Authenticate users or applications; resolve organisation context; apply organisation-aware access rules.	Input: credentials / access context. Output: authenticated identity and organisation scope.
2. Enterprise Memory Intelligence	Check for similar verified responses; decide whether an existing response can be reused; manage stored verified responses.	Input: validated request plus organisation context. Output: match decision and reusable response, or cache miss.
3. AI Gateway and Provider Management	Route cache misses to the selected LLM provider and handle provider request / response interaction.	Input: eligible request and provider selection. Output: provider response or error.
4. Audit Logging and Usage Analytics	Record relevant request outcomes and usage metadata; aggregate activity for reporting.	Input: gateway events and usage measures. Output: audit entries and analytics data.
5. Governance, Cost Monitoring and Dashboard	Expose monitoring information, cost-oriented views and central management capabilities.	Input: usage and audit aggregates. Output: dashboard views and governance visibility.

3.3 Data Flow / UML Diagrams

3.3.1 Context-Level Data Flow

At the context level, the enterprise application exchanges requests and responses with SentinelAI. The gateway communicates with external LLM providers only when a memory match does not satisfy the request, and it interacts with the enterprise memory and audit/usage store for lookup and recordkeeping.

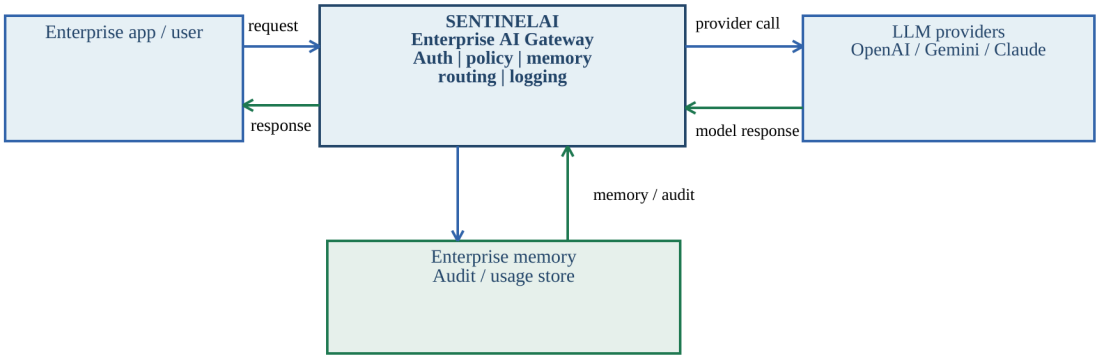


Figure 3.2 Context-level data flow diagram

3.3.2 Request Processing Activity Flow

The activity flow distinguishes the memory-hit and memory-miss paths. A request is authenticated and checked against policy before memory matching. A cache hit can be returned directly; a cache miss requires provider routing, response validation and storage. Both paths should record an appropriate audit event.

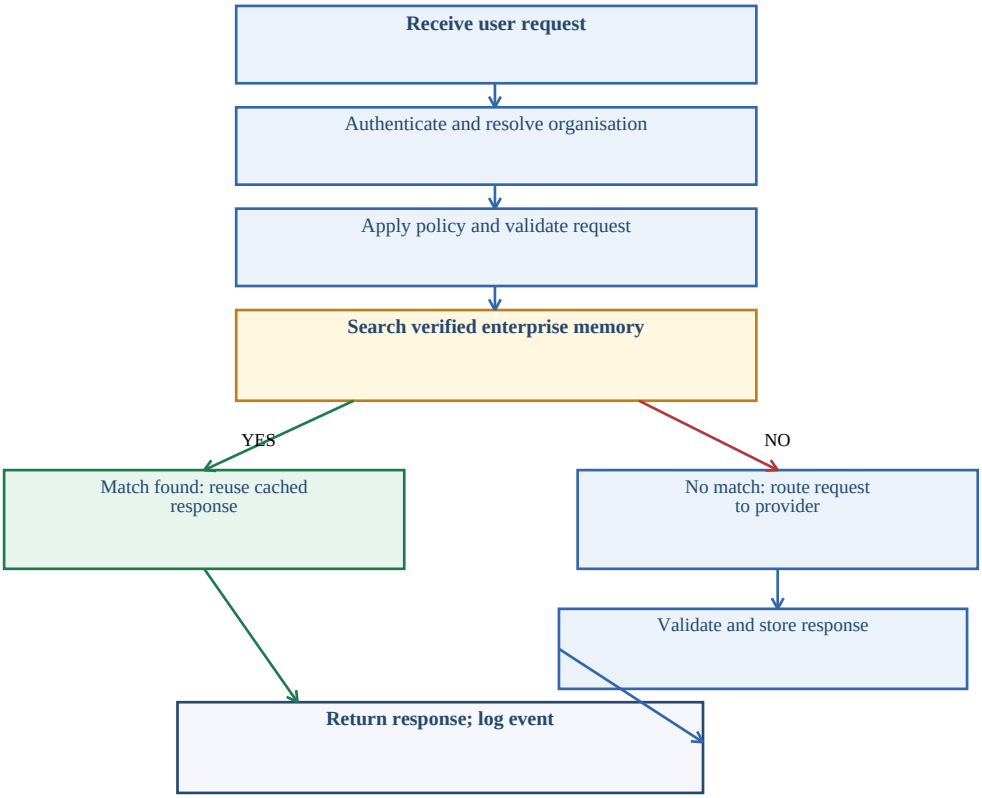


Figure 3.3 Request processing activity flow

3.3.3 Sequence of Gateway Interactions

The sequence diagram shows the planned interaction between the calling application, SentinelAI API, memory module, provider and audit/usage store. The provider-call and response-storage steps are conditional on the memory outcome and response verification policy.

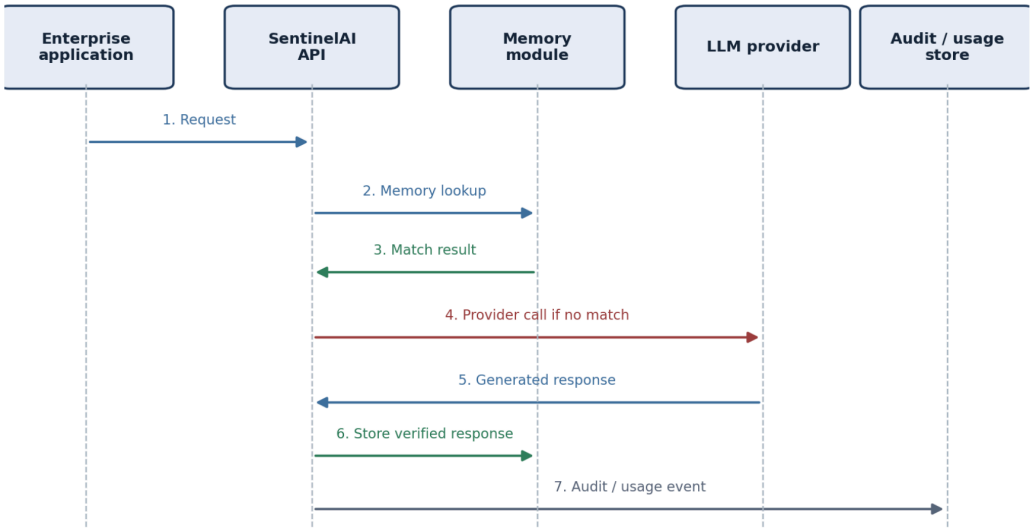


Figure 3.4 Sequence diagram for request, memory and provider handling

3.4 Threat Model and Attack Surface

The primary attack surface includes the API boundary, credentials and provider integrations, organisation scoping, the response-memory store, the administrative dashboard and the audit/analytics path. The review material references prompt injection as a relevant LLM risk [2]. The additional threats below are design risks that should be assessed during implementation; they are not recorded as attacks observed against a completed SentinelAI deployment.

Table 3.2 Threat model and attack surface

Threat / failure scenario	Potential impact	Design focus
Unauthorised API access	Unapproved users or applications submit prompts or consume provider quota.	Authentication, authorisation, organisation scope and rate limits.
Prompt injection or malicious instructions	Untrusted prompt content may influence model behaviour or downstream handling.	Input-risk handling, policy enforcement and response review.
Cross-organisation memory leakage	A response created for one tenant could be returned to another tenant.	Tenant-scoped memory keys, access checks and isolation tests.
Stale or incorrect verified response	An old or poorly verified response is reused where it is no longer appropriate.	Defined verification criteria, freshness metadata, expiry and invalidation.
Provider credential exposure	A leaked key may enable unauthorised provider use and unexpected charges.	Server-side secret storage, restricted permissions and safe logging.
Audit log exposure or alteration	Sensitive information may leak or accountability records may become unreliable.	Minimise logged content, restrict access and define retention/integrity rules.
Provider timeout or service failure	Requests may fail or remain unresolved.	Timeouts, error handling, clear status reporting and failure logs.

3.5 Security Controls

The following controls are recommended design requirements for the gateway. Their implementation status should be verified by future testing rather than assumed from the architecture alone.

Table 3.3 Proposed security controls

Control area	Proposed control / verification
Identity and access	Require authentication for protected endpoints and enforce role or organisation permissions. Test both authorised and unauthorised access paths.
Tenant isolation	Apply organisation scope consistently to memory lookup, response reuse, analytics and administrative views. Test attempts to access another organisation's records.
Input validation	Validate request schema, required fields, provider selection and size limits before routing.
Provider secrets	Keep provider credentials in server-side configuration; do not expose them to browser clients or return them in error messages.
Memory verification	Store only responses meeting a defined verification policy; retain metadata needed to judge organisation scope, freshness and provenance.
Audit and privacy	Log outcome, provider, timing and usage attributes required for governance while avoiding unnecessary storage of sensitive content.
Cost / abuse controls	Track provider calls and usage; define quotas or throttling policies before production deployment.
Failure handling	Handle provider errors and timeouts without returning malformed results or incorrectly marking unverified content as reusable.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 Data Collection

SentinelAI is not described as a model-training project in the supplied review material. Its operational data arises from AI requests passing through the gateway, responses returned by providers, memory lookups and the monitoring functions. No external training dataset is specified in the project references.

The following data elements are proposed for request handling and governance. The final schema and retention policy should be set during implementation, with collection limited to what is needed for operation, security and evaluation.

Table 4.1 Operational data elements

Data element	Purpose in the system	Consideration
Request identifier and timestamp	Correlate a request through gateway processing and measure elapsed time.	Use a unique identifier; timestamps should support ordering and analysis.
User / application and organisation context	Authenticate, authorise and apply tenant scope.	Avoid disclosing organisation-specific values across tenants.
Request content or matching representation	Process the request and check for a similar verified response.	Define whether raw prompts or a reduced representation is stored; protect sensitive content.
Provider and model selection	Route eligible requests and group usage by provider.	Do not log provider credentials.
Provider response and verification status	Validate output and identify responses eligible for reuse.	Define what “verified” means before storing a response as reusable.
Memory lookup outcome	Record hit/miss behaviour and support evaluation of reuse.	Record the outcome without assuming a hit is always safe or correct.
Usage and latency metadata	Evaluate request counts, response times and token/cost estimates.	Distinguish measured provider usage from estimates or configured price assumptions.
Audit event and request status	Support governance and diagnosis of completed or failed calls.	Define access, retention and privacy limits for logs.

4.2 Data Preprocessing

Preprocessing in SentinelAI refers to preparing each request and its context for access control, memory matching, provider routing and measurement. Since the review material does not specify the final implementation algorithm, the steps below are expressed as a proposed processing sequence rather than as completed code.

- Validate the request structure and required fields; reject malformed requests before they reach a provider.
- Resolve the caller identity and organisation context so that policy checks and memory lookup are scoped correctly.
- Apply request policies and select the provider or model according to the gateway configuration.
- Prepare a consistent representation for memory matching. The exact normalisation and similarity method, including its threshold, must be selected and documented during implementation.
- Accept a memory hit only when the response is relevant to the current request context, verified and permitted for reuse.
- For a cache miss, send the eligible request to the provider and inspect the returned status and content before storing a response as reusable.
- Capture the timing, provider, usage and outcome information required for later evaluation, while minimising sensitive data in logs.

4.2.1 Algorithms / Methodology

The planned request methodology can be expressed as a high-level algorithm. This flow captures the logic shown in the review diagrams and does not specify a particular embedding model, similarity metric or verification algorithm.

INPUT: request R, caller identity U, organisation O

1. Authenticate U and resolve O.
2. If authentication fails, reject R and record an access outcome.
3. Apply organisation policy and validate R.
4. Search the organisation-scoped memory for a similar verified response.
5. If a suitable verified match is found, retrieve the response, record a memory hit and return the response.
6. Otherwise, route R to the selected LLM provider; receive the response or error; validate the response and reuse eligibility; store eligible responses with appropriate metadata; return the response or a clear error status.
7. Record usage, latency, status and audit metadata for the request.
8. Update analytics and cost-monitoring aggregates.

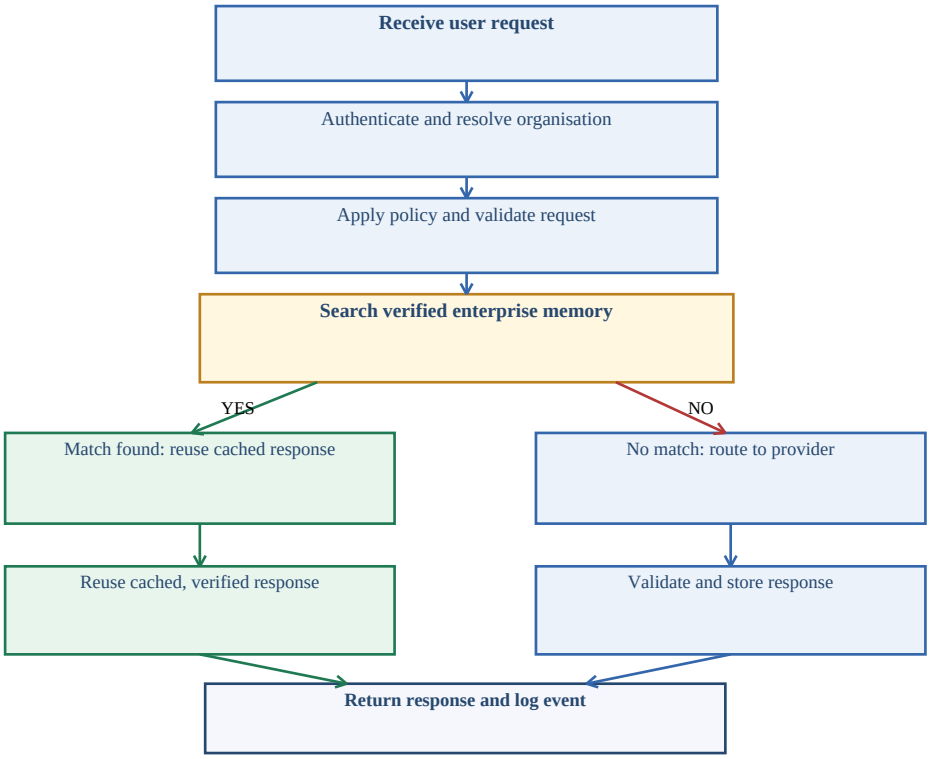


Figure 4.1 Enterprise Memory Intelligence decision flow (conceptual)

4.2.2 Results and Discussion

The available review material reports planning progress and expected outcomes, but it does not include measured performance results from a completed implementation. At Review 1, the literature survey, problem identification, requirement analysis/methodology and proposed architecture/core modules were marked complete. Implementation, testing and final evaluation were not marked complete.

The proposed benefits are a reduction in redundant LLM calls, token consumption and operating cost; faster responses for repeated or similar requests; improved response consistency when verified outputs can be reused; and centralised usage visibility. These benefits remain expected outcomes until experiments are run. No percentage reduction, latency figure, token savings or accuracy measurement is stated in the reference materials, so numerical performance claims are not made in this report.

Table 4.2 Preliminary evaluation criteria

Evaluation criterion	Method to evaluate after implementation	Evidence currently available
Memory reuse	Submit controlled repeated / similar prompts and calculate the memory-hit rate.	Design requirement only; no measured hit rate reported.
Provider-call reduction	Compare provider call counts with memory reuse enabled and disabled for the same test set.	Expected benefit; no call-count measurements reported.
Response latency	Measure elapsed time for cache hits and provider-generated responses using the same environment.	Expected faster repeat response; no latency values reported.
Token and cost impact	Collect provider usage metrics and apply a documented price basis for the comparison.	Expected reduction; no measured token/cost totals reported.
Response suitability	Review whether a memory match is relevant and whether the stored response meets the defined verification policy.	Verification method and test results not yet specified.
Security and tenant isolation	Test denied access, organisation boundaries, prompt-risk handling and audit completeness.	Security considerations identified; testing marked pending.

Discussion.

Enterprise Memory Intelligence is the project's main differentiating element, but it must be implemented with care: a false or stale memory match can harm response quality, and a memory lookup that ignores organisation scope can expose information between tenants. Therefore, performance evaluation should be paired with correctness and security testing rather than optimising call reduction alone.

CHAPTER 5

CONCLUSION

5.1 Conclusion

SentinelAI proposes a centralized gateway for enterprise applications that use Large Language Models. It places a managed layer between applications and providers such as OpenAI, Google Gemini and Anthropic Claude, combining authentication, organisation-based management, policy enforcement, provider routing, audit logging, usage analytics and cost monitoring.

The central contribution is Enterprise Memory Intelligence, which checks for similar verified responses before requesting a new provider output. A suitable match is intended to be reused, while a miss proceeds through the provider router and response-validation path. This design aims to reduce redundant calls and improve the management and consistency of AI usage across applications.

The literature survey, problem identification, requirement analysis and methodology, and proposed architecture and core modules were recorded as complete at Review 1. Implementation, testing and final evaluation still need to be supported by measured evidence. The next stage is to implement the modules, define memory verification and similarity rules, and evaluate both effectiveness and security against a documented baseline.

5.2 Future Work

- Complete end-to-end integration of authentication, organisation management, Enterprise Memory Intelligence, provider routing, audit logging, analytics and the dashboard.
- Define the memory matching method, similarity threshold, verification criteria, freshness metadata, expiry rules and invalidation process. Record provenance for reusable responses.
- Test organisation isolation across memory, audit logs and analytics. Include unauthorised access, prompt injection, malformed requests, provider timeouts and error recovery.
- Compare memory-enabled and memory-disabled runs with the same prompt set and provider settings. Measure hit rate, provider-call reduction, latency, token use, cost and response suitability.
- Document the environment, model versions and cost assumptions. Use measured results to refine the policies and repeat the tests before production deployment.

REFERENCES

[1] Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS 2020.

[2] Willison, S. (2023). “Prompt Injection Attacks Against Large Language Models.” Personal Blog / ACL Workshop, as listed in the project review material.

[3] Chase, H. (2023). “LangChain: Building LLM Applications.” GitHub / LangChain Documentation, as listed in the project review material.

[4] OpenAI. (2023). “GPT-4 Technical Report.” arXiv:2303.08774.

[5] Anthropic. (2022). “Constitutional AI: Harmlessness from AI Feedback.” arXiv:2212.08073.

[6] OWASP GenAI Security Project. (2026). “OWASP GenAI LLM Top 10.” https://genai.owasp.org/llm-top-10/.

[7] Tabassi, E. (2023). “Artificial Intelligence Risk Management Framework (AI RMF 1.0).” NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1.

[8] NIST. (2024). “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile.” NIST AI 600-1.

[9] Django Software Foundation. (2026). “Security in Django.” Django Documentation. https://docs.djangoproject.com/en/6.1/topics/security/.

[10] Google. (2026). “Gemini API Documentation.” Google AI for Developers. https://ai.google.dev/gemini-api/docs.

APPENDIX A

SYSTEM REQUIREMENTS AND EVALUATION PLAN

A.1 Hardware Requirements

Table A.1 Hardware requirements

S. No.	Hardware component	Requirement stated in review material
1	Processor	Intel Core i5 or above
2	Memory	8 GB RAM minimum
3	Storage	256 GB SSD
4	Connectivity	Stable Internet connection

A.2 Software Requirements

Table A.2 Software requirements

S. No.	Software / tool	Role stated or implied by the project design
1	Windows 11	Operating system listed in the zeroth review.
2	Python 3.13+	Programming language for backend and gateway logic.
3	Django and Django REST Framework	Backend application and API layer.
4	React.js / Bun	Frontend application and development workflow.
5	PostgreSQL / SQLite	Database options identified in review material; primary deployment choice remains to be finalised.
6	Postman	API request and endpoint testing.
7	Git and GitHub	Version control and source collaboration.
8	Gemini API and multiple LLM provider APIs	External model integration; OpenAI, Gemini and Claude are named in the architecture.

A.3 Minimum Evaluation Record

For each evaluation run, record the test date, environment, prompt set, organisation context, provider/model, whether memory reuse was enabled, request count, memory hits, provider calls, response time, usage metrics and validation outcome. Use the same test set and equivalent settings when comparing memory enabled and disabled. Report measured values only and clearly identify any cost calculation assumptions.

APPENDIX B

PROJECT SCREENSHOTS

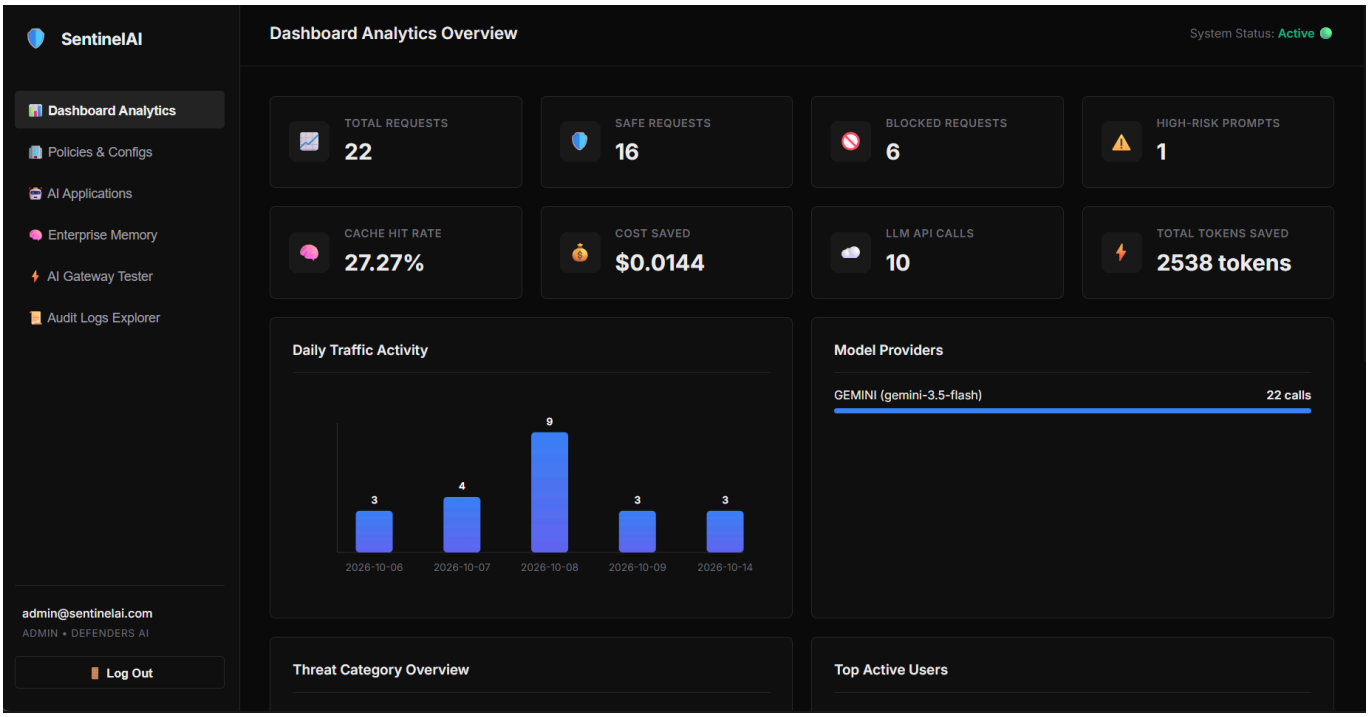


Figure B.1 Dashboard analytics overview

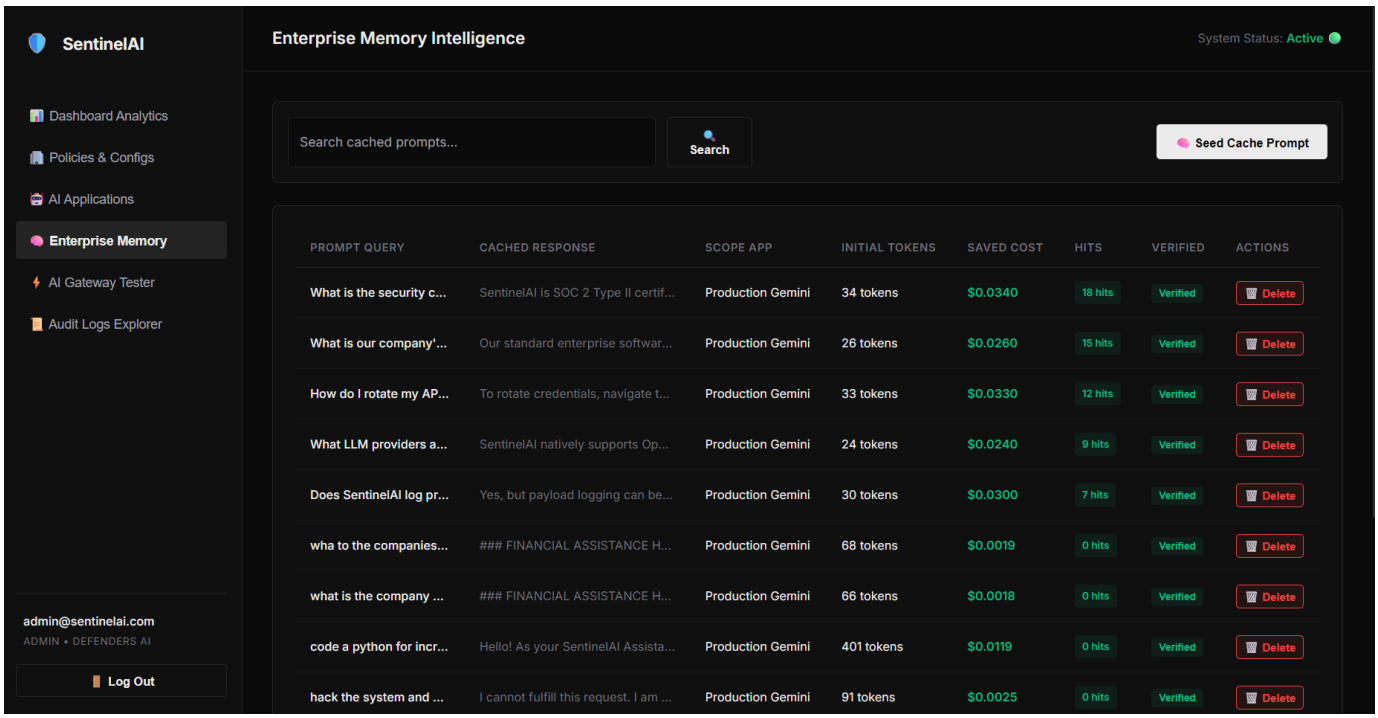


Figure B.2 Enterprise Memory Intelligence cache records

PROJECT SCREENSHOTS (CONTINUED)

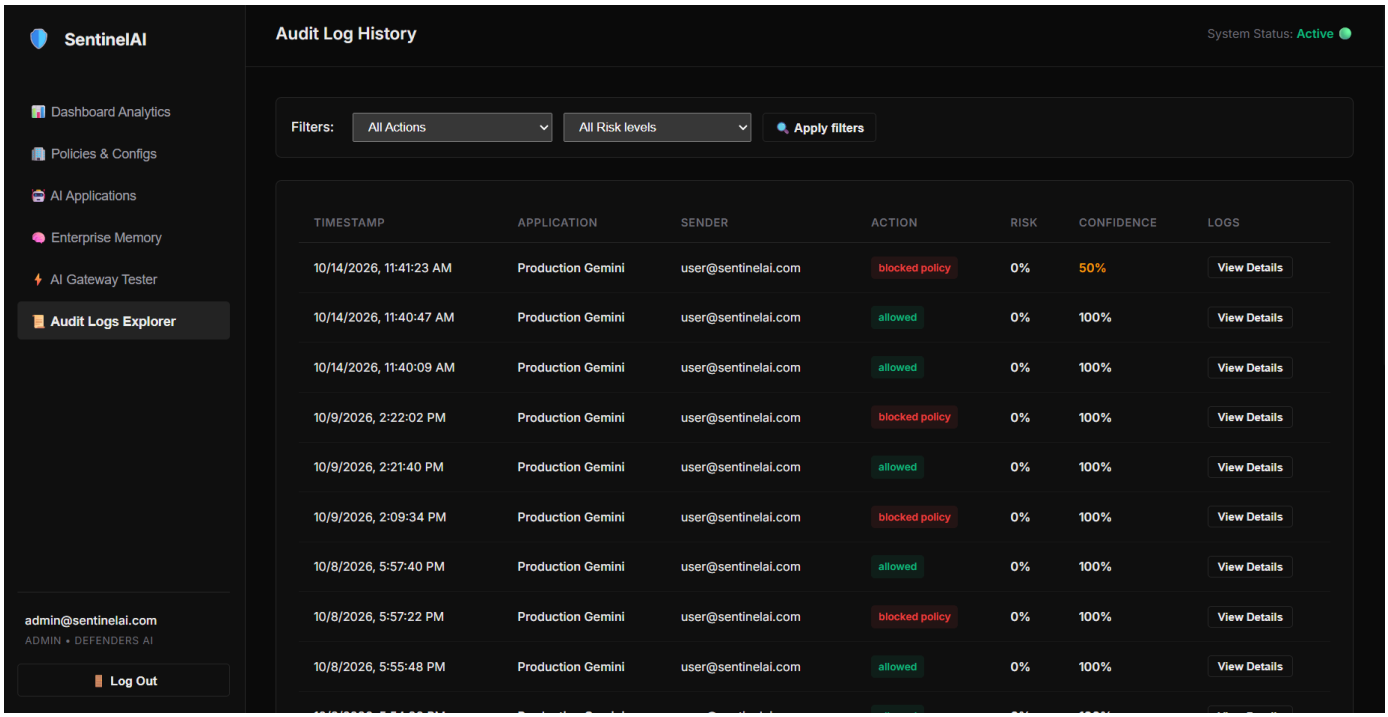


Figure B.3 Audit log history

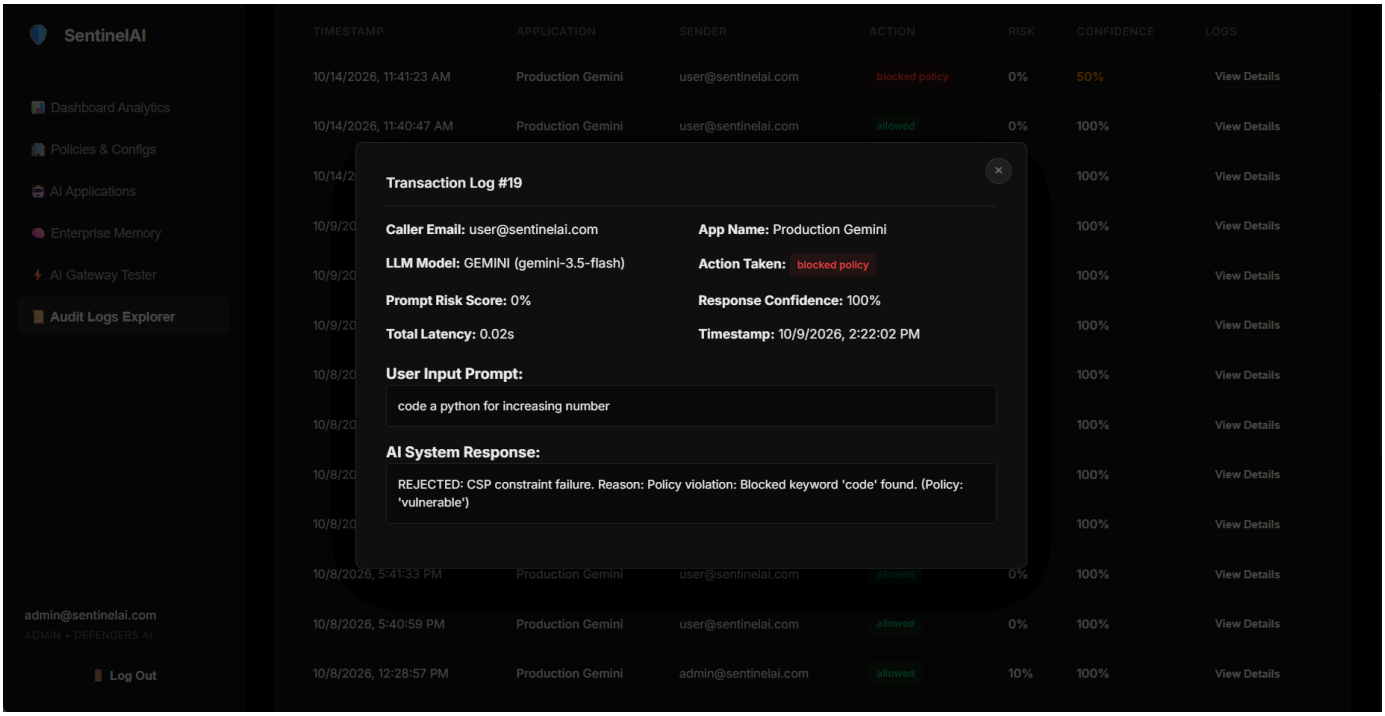


Figure B.4 Audit transaction details

PROJECT SCREENSHOTS (CONTINUED)

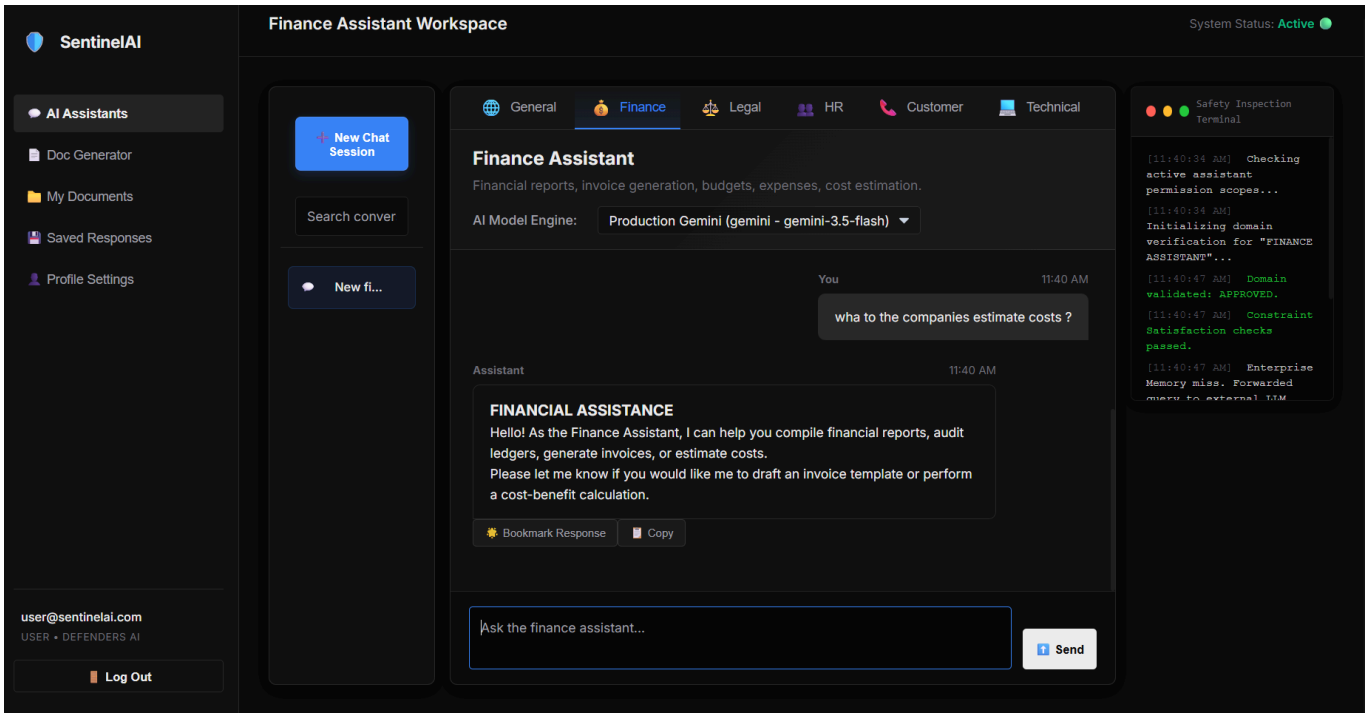


Figure B.5 Finance assistant workspace

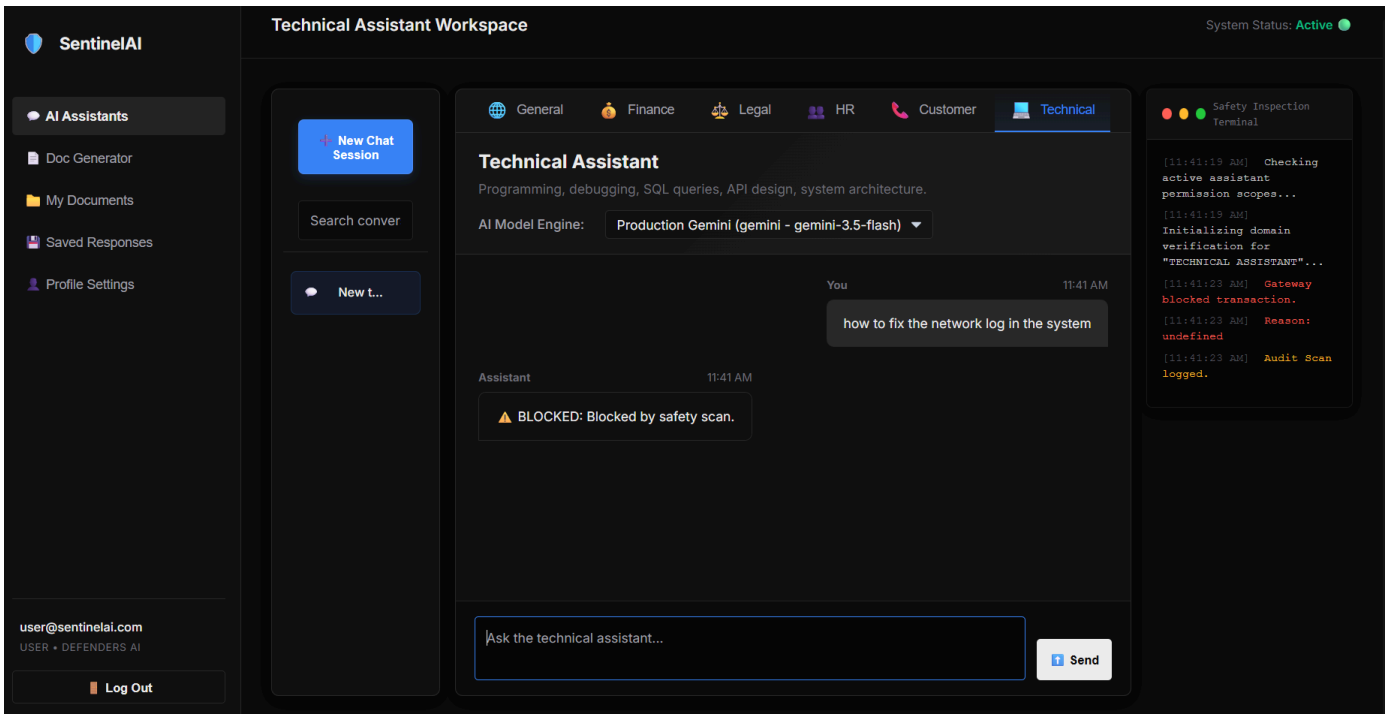


Figure B.6 Technical assistant workspace